

PCOD

MÔ TẢ VÀ HƯỚNG DẪN ĐỊNH DẠNG

PCOD Hybrid Database 0.2 - Bản tiếng Việt

Tên định dạng	PCOD Hybrid Database
Phiên bản định dạng	0.2
Phiên bản tham chiếu	PCOD 0.6.23
Tác giả / Chủ sở hữu	SpiderPawn Việt Nam
Giấy phép tài liệu	MIT License
Bản tài liệu	1.0 - 2026

Tài liệu mô tả kỹ thuật và hướng dẫn triển khai, dành cho nhà phát triển, công cụ AI và người quản trị dữ liệu.

Mục lục

Mục lục được trình bày tĩnh để bảo đảm hiển thị ổn định trên mọi trình đọc PDF.

- 1 Phạm vi, đối tượng và giấy phép**
- 2 Tổng quan trong 60 giây**
- 3 Cấu trúc thư mục .pcod**
- 4 manifest.properties**
- 5 games.pci - chỉ mục cố định**
- 6 names.pcn - từ điển tên**
- 7 games.pcg - luồng payload**
- 8 Hai codec payload**
- 9 Khôi phục PGN hoàn chỉnh**
- 10 Âm thanh và H2 sidecar**
- 11 Quy trình đọc an toàn**
- 12 Quy trình import và crash recovery**
- 13 Sửa, xóa mềm và compact**
- 14 Validation và an toàn dữ liệu**
- 15 Chuyển đổi định dạng**
- 16 Python skeleton đọc index**
- 17 Cheat sheet và xử lý lỗi**
- 18 Tương thích tương lai**
- 19 Giấy phép MIT**

1. Phạm vi, đối tượng và giấy phép

Tài liệu này mô tả định dạng lưu trữ PCOD Hybrid Database phiên bản 0.2, được đối chiếu với phần mềm PCOD 0.6.23. Mục tiêu là giúp một nhà phát triển mới, một công cụ kiểm tra dữ liệu hoặc một trợ lý AI có thể hiểu cấu trúc, đọc dữ liệu và đánh giá tính toàn vẹn mà không phải suy đoán từ phần mở rộng tệp.

1.1. Đối tượng sử dụng

- **Nhà phát triển reader:** cần đọc, tìm kiếm hoặc xuất PGN mà không thay đổi database.
- **Nhà phát triển writer/importer:** cần ghi dữ liệu có giao dịch, rollback và kiểm thử crash.
- **Công cụ kiểm tra:** cần phát hiện tệp bị cắt, CRC sai, ID tên không tồn tại hoặc payload lỗi.
- **Công cụ AI:** cần có một tài liệu có thẩm quyền để không tự suy đoán byte layout.
- **Người quản trị dữ liệu:** cần sao lưu, di chuyển, khôi phục hoặc compact database đúng cách.

1.2. Thông tin tác giả và quyền sử dụng

Mục	Thông tin
Tác giả / Chủ sở hữu bản quyền	SpiderPawn Việt Nam
Năm bản quyền	2026
Giấy phép tài liệu	MIT License
Phạm vi giấy phép	Tài liệu mô tả và các tệp đặc tả đi kèm
Không mặc nhiên áp dụng	Mã nguồn, SDK, logo, nhãn hiệu hoặc thành phần phần mềm khác nếu không có thông báo riêng

Lưu ý pháp lý: Bản MIT trong tài liệu này cấp quyền cho chính tài liệu và đặc tả kèm theo. Việc dùng phần mềm PCOD, mã nguồn hoặc tài sản thương hiệu vẫn phải tuân theo giấy phép riêng của từng thành phần.

1.3. Nguyên tắc bắt buộc

- `.pcod` là một thư mục bundle, không phải một file đơn lẻ.
- `games.pci` có header 64 byte và record cố định 56 byte; không được đổi kích thước trong format 0.2.
- Game ID bằng vị trí record + 1, bắt đầu từ 1 và không được tái sử dụng.
- Xóa ván là soft delete bằng flag; không xóa record vật lý.
- `games.pcg` là append-only; sửa ván tạo payload mới và đổi con trỏ index.
- Phải kiểm tra CRC32 của payload trước khi decode.
- Reader phải xử lý mixed-endian chính xác.
- Chỉ viết dữ liệu khi đã có recovery marker, flush/force và kiểm thử crash.
- Nếu `formatVersion` khác `0.2`, phải từ chối hoặc dùng reader tương thích khác.

2. Tổng quan trong 60 giây

PCOD Hybrid 0.2 tách dữ liệu thành ba lớp để vừa tìm kiếm nhanh, vừa giảm lặp tên, vừa bảo toàn nội dung PGN đầy đủ.

```
Metadata tìm kiếm nhanh -> games.pci
Tên lặp lại             -> names.pcn
Nội dung đầy đủ của ván -> games.pcg
```

2.1. Luồng đọc một ván

```

Game ID
|
v
games.pci: đọc record 56 byte
|
+--> names.pcn: đổi whiteId/blackId/eventId/siteId thành chuỗi
|
+--> games.pcg: đọc payloadOffset + payloadLength
|
+--> kiểm tra CRC32
+--> decode MOVE16 hoặc PGN_DEFLATE
+--> dựng lại PGN hoàn chỉnh

```

2.2. Luồng import

```

PGN -> ParsedGame -> name IDs + packed metadata
-> append payload vào games.pcg
-> append record chưa commit vào games.pci
-> atomic save names.pcn
-> commit gameCount trong header games.pci

```

Điểm commit: Giá trị `gameCount` tại offset 16 của header `games.pci` là điểm commit cuối cùng. Record vật lý nằm sau count nhưng chưa được commit phải bị bỏ khi recovery.

3. Cấu trúc thư mục .pcod

```

MyDatabase.pcod/
|-- manifest.properties      bắt buộc
|-- games.pci               bắt buộc
|-- names.pcn               có sau khi có dữ liệu tên
|-- games.pcg               bắt buộc
|-- extras.mv.db            tùy chọn, H2 sidecar
|-- sound/                  tùy chọn
|   |-- <sha256>.mp3
|   |-- <sha256>.flac
|   |-- <sha256>.ogg
|-- recovery/               xuất hiện khi có giao dịch đang dở
|   |-- import.properties
|   |-- update.properties

```

Thành phần	Vai trò	Bắt buộc	Sao lưu
manifest.properties	Nhận dạng format, version và codec	Có	Có
games.pci	Chỉ mục cố định, Game ID, metadata nóng	Có	Có
names.pcn	Từ điển Player/Event/Site	Khi có tên	Có
games.pcg	Payload hoàn chỉnh của từng ván	Có	Có
extras.mv.db	Lịch sử import, ghi chú người dùng	Không	Nên
sound/	Âm thanh được movetext tham chiếu	Không	Có nếu sử dụng
recovery/	Marker phục hồi giao dịch	Tạm thời	Giữ nguyên khi có sự cố

3.1. File tạm khi ghi atomic

```

manifest.properties.tmp
names.pcn.tmp
recovery/import.properties.tmp
recovery/update.properties.tmp

```

Không được coi file `.tmp` là dữ liệu đã commit. Khi sao lưu trong lúc ứng dụng đang ghi, nên dừng ứng dụng hoặc dùng snapshot nhất quán ở mức filesystem.

3.2. Thành phần không thuộc database

Extension, theme, bộ quân cờ, font và language pack nằm trong vùng cấu hình ứng dụng. Chúng không phải thành phần của PCOD Hybrid 0.2 và không được đưa vào logic đọc database.

4. manifest.properties

Đây là Java Properties file dùng để nhận dạng database trước khi đọc các file binary.

```
format=PCOD-HYBRID
formatVersion=0.2
databaseId=550e8400-e29b-41d4-a716-446655440000
createdAt=2026-06-14T10\:20\:30.123Z
indexRecordSize=56
payloadCodecs=MOVE16_V1,PGN_DEFLATE_V1
```

Key	Giá trị / ý nghĩa
format	Phải bằng `PCOD-HYBRID`
formatVersion	Phải bằng `0.2` đối với tài liệu này
databaseId	UUID nhận dạng database
createdAt	Thời điểm tạo ISO-8601
indexRecordSize	56
payloadCodecs	MOVE16_V1,PGN_DEFLATE_V1

4.1. Quy tắc tương thích

- Từ chối mở nếu `format` khác `PCOD-HYBRID`.
- Từ chối mở nếu `formatVersion` khác `0.2`.
- Từ chối mở nếu `indexRecordSize` có mặt nhưng khác `56`.
- Từ chối giá trị số sai cú pháp hoặc vượt giới hạn.
- Bỏ qua key chưa biết để dành chỗ mở rộng, trừ khi key đó làm thay đổi boundary hoặc codec bắt buộc.

5. games.pci - chỉ mục cố định

`games.pci` là hot index cho phép truy cập record theo Game ID với phép tính offset trực tiếp. Toàn bộ file dùng little-endian.

```
recordOffset(gameId) = 64 + (gameId - 1) * 56
expectedFileLength = 64 + committedGameCount * 56
```

5.1. Header 64 byte

Offset	Size	Kiểu	Nội dung
0	4	int32 LE	magic numeric 0x50434931 (PCI1)
4	2	uint16 LE	version = 1
6	2	uint16 LE	record size = 56
8	4	int32 LE	header size = 64
12	4	int32 LE	reserved, hiện 0
16	8	int64 LE	committed game count

Offset	Size	Kiểu	Nội dung
24	8	int64 LE	reserved, thường khởi tạo 1
32	8	int64 LE	Unix epoch milliseconds khi tạo
40	24	bytes	reserved

Magic và byte order: Constant Java là `0x50434931`, nhưng file ghi little-endian. Hãy decode int32 LE rồi so sánh giá trị; không dò chuỗi ASCII trực tiếp trong raw bytes.

5.2. Record 56 byte

Offset	Size	Kiểu	Field
0	8	int64 LE	payloadOffset
8	4	int32 LE	payloadLength
12	4	int32 LE	whiteId
16	4	int32 LE	blackId
20	4	int32 LE	eventId
24	4	int32 LE	siteId
28	4	uint32 LE	packedDate
32	4	uint32 LE	packedElo
36	4	uint32 LE	packedMeta
40	4	int32 LE	plyCount
44	8	uint64 LE	semanticHash
52	4	uint32 LE	payloadCrc

5.3. packedDate

Bits	Mask	Ý nghĩa
0-13	0x00003FFF	Năm; 0 = không biết
14-17	0x0003C000	Tháng; 0 = không biết
18-22	0x007C0000	Ngày; 0 = không biết
23-24	0x01800000	Precision: 0 none, 1 year, 2 month, 3 day
25-31	-	Reserved

```

year    = packedDate & 0x3FFF
month   = (packedDate >>> 14) & 0x0F
day     = (packedDate >>> 18) & 0x1F
precision = (packedDate >>> 23) & 0x03

```

Khi payload có exact Date text, reader đầy đủ phải ưu tiên Date trong payload. `packedDate` chủ yếu phục vụ tìm kiếm nhanh và fallback.

5.4. packedElo và packedMeta

```
whiteEloRaw = packedElo & 0xFFFF
blackEloRaw = (packedElo >>> 16) & 0xFFFF
```

Bits packedMeta	Ý nghĩa
0-9	ECO packed
10-11	Result code
12	FLAG_DELETED
13	FLAG_COMMENTS
14	FLAG_VARIATIONS
15	FLAG_START_FEN
16	FLAG_MOVE16
17-31	Reserved

Result code	Kết quả
0	* hoặc không xác định
1	1-0
2	0-1
3	1/2-1/2

ECO packed

0 là không biết. Các giá trị 1..500 ánh xạ A00..E99 theo công thức:

```
packed = 1 + (letter - 'A') * 100 + number
n      = packed - 1
letter = 'A' + n / 100
number = n % 100
```

5.5. plyCount, semanticHash và payloadCrc

- **plyCount:** MOVE16 lưu số move chính xác; PGN_DEFLATE chỉ là ước lượng main line.
- **semanticHash:** FNV-1a 64-bit dùng nhận dạng nội dung, không phải checksum và không phải chữ ký bảo mật.
- **payloadCrc:** CRC32 tiêu chuẩn của toàn bộ byte payload; phải kiểm tra trước khi decode.

```
trim(White) + U+001F + trim(Black) + U+001F + trim(Date)
+ U+001F + trim(Result) + U+001F
+ movetext đã gom whitespace và trim
```

5.6. Soft delete và chiều dài file

```
packedMeta |= 1 << 12
```

- Nếu file ngắn hơn `64 + gameCount \* 56`: database bị cắt, phải báo lỗi.
- Nếu file dài hơn chiều dài đã commit: phần dư là record chưa commit hoặc trailing bytes; recovery có thể truncate.
- Không tính gameCount bằng cách lấy file size chia cho 56.
- Game ID đã xóa không được tái sử dụng.

6. names.pcn - từ điển tên

`names.pcn` loại bỏ lặp lại Player, Event và Site. Các field cố định dùng big-endian; varint dùng nhóm 7 bit thấp trước.

6.1. Header và thứ tự section

Offset	Size	Kiểu	Nội dung
0	4	int32 BE	magic 0x50434E31, raw ASCII PCN1
4	2	uint16 BE	version = 1
6	2	uint16 BE	reserved = 0

PLAYER
EVENT
SITE

6.2. Cấu trúc mỗi section

```
int32 BE count
repeat count times:
    unsigned-varint stableId
    unsigned-varint commonPrefixLength
    unsigned-varint suffixByteLength
    byte[suffixByteLength] UTF-8 suffix
```

6.3. Front coding và UTF-16

```
value = previous.substring(0, commonPrefixLength) + UTF8(suffix)
```

Điểm dễ sai nhất: `commonPrefixLength` là số UTF-16 code units theo Java `String.length()`, không phải số byte UTF-8, code point hay grapheme. Ký tự supplementary/emoji có thể chiếm hai unit.

6.4. Stable ID không theo vị trí vật lý

- Mỗi entry lưu `stableId` rõ ràng.
- Các entry được sắp theo tên để front-code, không sắp theo ID.
- Không giả định entry thứ N có ID N.
- ID 0 không đại diện cho tên thật; lookup sai thường trả `?`.
- Index record chỉ có ý nghĩa cùng đúng file `names.pcn` của database đó.

6.5. Chuẩn hóa chống trùng

- Null/rỗng thành `?`.
- Trim và gom whitespace.
- Unicode NFD.
- Xóa combining marks.
- Uppercase theo Locale.ROOT.
- Ký tự ngoài [A-Z0-9] thành dấu cách.
- Trim và gom khoảng trắng lần cuối.

Hai chuỗi khác dấu hoặc khác chữ hoa/thường có thể dùng chung ID. Chuỗi được thêm đầu tiên được giữ làm giá trị hiển thị.

6.6. Ghi atomic

Writer phải ghi toàn bộ dictionary vào `names.pcn.tmp`, flush/close, rồi atomic replace nếu filesystem hỗ trợ. Không sửa một phần dictionary tại chỗ.

7. games.pcg - luồng payload

`games.pcg` không có global header. Đây là chuỗi payload nối tiếp; boundary chỉ đến từ `payloadOffset` và `payloadLength` trong index.

- Không quét tuần tự `games.pcg` để suy ra danh sách ván.
- Payload cũ có thể trở thành orphan sau update.
- Không giả định mọi byte đều reachable.
- CRC được tính trên toàn bộ byte payload mà record trở tới.

7.1. Header của từng payload PCG1

Offset	Size	Kiểu	Nội dung
0	4	int32 BE	magic 0x50434731, ASCII PCG1
4	1	uint8	payload version = 1
5	1	uint8	codec: 1 MOVE16, 2 PGN_DEFLATE
6	2	uint16 BE	reserved = 0

```
string exactDate
string round
string fen
unsigned-varint extraTagCount
repeat extraTagCount:
    string tagName
    string tagValue
codec-specific body
```

7.2. String và custom tags

```
unsigned-varint utf8ByteLength
byte[utf8ByteLength] UTF-8
```

Null được ghi như chuỗi rỗng. Standard tags như Event, Site, Date, Round, White, Black, Result, Elo, ECO, FEN, SetUp và PlyCount không được lặp trong extra tags.

8. Hai codec payload

8.1. MOVE16_V1

MOVE16 lưu nước đi theo mã 16 bit và dựng lại SAN từ trạng thái bàn cờ. Chỉ dùng khi main line có thể round-trip an toàn.

- Starting FEN hợp lệ hoặc vị trí chuẩn.
- Toàn bộ mainline SAN parse được.
- Không có comment, semicolon comment, variation hoặc NAG.
- Không có cấu trúc khiến parser thất bại.
- Nếu bất kỳ điều kiện nào không đạt, phải fallback PGN_DEFLATE.

```
unsigned-varint moveCount
repeat moveCount:
    uint8 lowByte
    uint8 highByte
```

Bits	Ý nghĩa
0-5	from square 0..63
6-11	to square 0..63
12-14	promotion code
15	không dùng

Promotion	Quân
-----------	------

0	Không promotion
1	Knight
2	Bishop
3	Rook
4	Queen

```
code = (from & 63) | ((to & 63) << 6) | (promotionCode << 12)
```

Square mapping: Format chỉ nói giá trị 0..63; implementation ngoài PCOD phải xác minh convention bằng test vector e2-e4, g1-f3, nhập thành và promotion. Không phát hành writer trước khi round-trip khớp.

Dựng lại SAN

8. Khởi tạo bàn cờ từ FEN hoặc vị trí chuẩn.
9. Unpack from/to/promotion.
10. Sinh SAN từ trạng thái hiện tại.
11. Apply move.
12. Thêm số nước.
13. Gắn Result từ index.

8.2. PGN_DEFLATE_V1

Codec này bảo toàn movetext UTF-8 khi dữ liệu có comments, variations, NAGs, graphical annotations, sound directive, SAN phức tạp/lỗi hoặc FEN không hợp lệ.

```
unsigned-varint rawUtf8Length
unsigned-varint compressedLength
byte[compressedLength] zlib-compressed data
```

- Container là zlib stream, không phải raw DEFLATE và không phải gzip.
- Compressor tham chiếu dùng Java Deflater với BEST_SPEED.
- Reader phải giới hạn raw length để chống decompression bomb.
- compressedLength phải nằm trong payload boundary.
- Số byte sau inflate phải bằng rawUtf8Length.

9. Khôi phục PGN hoàn chỉnh

Reader kết hợp index, dictionary và payload để dựng lại PGN. Thứ tự tag chuẩn là:

```
Event
Site
Date
Round
White
Black
Result
WhiteElo      nếu có
BlackElo      nếu có
ECO           nếu có
SetUp         nếu có FEN
FEN           nếu có
custom tags   theo thứ tự payload
```

9.1. Pseudocode

```

record = readIndex(gameId)
assert !record.deleted
payload = read(games.pcg, record.payloadOffset, record.payloadLength)
assert CRC32(payload) == record.payloadCrc
decoded = decodePayload(payload, resultText(record.resultCode))

tags.Event = dictionary.EVENT[record.eventId]
tags.Site = dictionary.SITE[record.siteId]
tags.Date = decoded.date != "" ? decoded.date : unpackDate(record)
tags.Round = decoded.round != "" ? decoded.round : "?"
tags.White = dictionary.PLAYER[record.whiteId]
tags.Black = dictionary.PLAYER[record.blackId]
tags.Result = resultText(record.resultCode)

append optional Elo/EC0/FEN and custom tags
write blank line + movetext

```

9.2. Escaping và annotation

```

\ -> \\
" -> \"

```

Loại	Dạng lưu
Bình luận	{...} hoặc ;...
NAG	\$<number>
Biến phụ	RAV trong (...)
Diagram	comment bắt đầu # hoặc [%diagram]
Mũi tên	[%cal <items>]
Tô ô	[%csl <items>]
Âm thanh	[%pcod_sound filename.ext]

Mã màu annotation: G green, R red, Y yellow, B blue, O orange. Arrow item có dạng `Ge2e4`; square item có dạng `Yd4`.

10. Âm thanh và H2 sidecar

10.1. Âm thanh

Âm thanh không được nhúng vào `games.pcg`. Movetext chỉ chứa reference, còn file thật nằm trong `sound/`.

```
1. e4 [%pcod_sound <sha256>.mp3]} e5
```

- Mỗi move node tối đa một sound reference.
- File nguồn tối đa 50 MiB.
- Chỉ chấp nhận mp3, flac, ogg.
- Tên file là SHA-256 nội dung + extension lowercase.
- Reader chỉ nhận basename an toàn; chặn `..` và path traversal.
- Thiếu file âm thanh không làm core reader crash; chỉ báo missing asset.

10.2. extras.mv.db

H2 sidecar là tùy chọn; core vẫn phải đọc được ván nếu file này thiếu hoặc driver H2 không có.

```
jdbc:h2:file:<database>/extras;AUTO_SERVER=FALSE;DB_CLOSE_ON_EXIT=TRUE
```

Bảng	Mục đích
SOURCE_IMPORT	Lịch sử nguồn import, số ván nhập/bỏ qua, thời điểm

Bảng	Mục đích
USER_NOTE	Ghi chú, rating và thời điểm sửa theo Game ID

- Không dùng H2 làm nguồn duy nhất của dữ liệu ván.
- Không yêu cầu H2 để export PGN core.
- Schema mới trong sidecar phải được version hóa riêng.

11. Quy trình đọc an toàn

14. Xác nhận root có hậu tố `.pcod`` và là thư mục.
15. Đọc Java Properties; kiểm tra format, formatVersion và record size.
16. Nếu có recovery marker, chạy recovery bằng code tương thích hoặc từ chối mở read-write.
17. Đọc header `games.pci`` little-endian và lấy committed gameCount.
18. Kiểm tra file index không bị cắt; xử lý trailing bytes theo chế độ mở.
19. Đọc toàn bộ `names.pcn`` với fixed fields big-endian, varint và front coding UTF-16.
20. Với mỗi Game ID hợp lệ, đọc đúng record 56 byte.
21. Nếu deleted và người dùng không yêu cầu, bỏ qua.
22. Kiểm tra offset/length không âm, không overflow và nằm trong `games.pcg``.
23. Đọc payload bằng pread, kiểm tra CRC32 trước decode.
24. Kiểm tra PCG1, version, codec, string/tag boundary.
25. Decode codec với giới hạn tài nguyên.
26. Lookup stable IDs và dựng lại PGN.

11.1. Chế độ read-only khuyến nghị

Khi mục tiêu chỉ là kiểm tra hoặc xuất dữ liệu, mở file read-only và không tự truncate trailing bytes. Hãy báo cảnh báo để người dùng chạy công cụ recovery chính thức.

12. Quy trình import và crash recovery

12.1. Marker import

```
recovery/import.properties
gameCount=<committed count trước import>
payloadLength=<games.pcg length trước import>
players=<player count trước import>
events=<event count trước import>
sites=<site count trước import>
```

Marker đời đầu có thể thiếu ba dictionary counts. Reader vẫn rollback index và payload; unused names còn lại thường vô hại.

12.2. Thứ tự import an toàn

27. Ghi marker atomically và force.
28. Với từng ván, checkpoint payload length và dictionary counts.
29. Encode payload và cấp name IDs.
30. Append payload.
31. Append index record trong vùng chưa commit.
32. Nếu ván lỗi, truncate payload và rollback names của riêng ván.
33. Force games.pcg.
34. Force record data trong games.pci.
35. Atomic save names.pcn.
36. Ghi committed gameCount ở offset 16 và force.
37. Xóa marker.

12.3. Recovery khi mở

```
truncate games.pci về gameCount
truncate games.pcg về payloadLength
rollback names về counts nếu marker có counts
atomic save names.pcn
xóa marker
```

Tại sao count phải cuối cùng: Nếu count chưa tăng, record mới bị coi là chưa commit. Nếu count đã tăng, mọi payload/index/dictionary mà record tham chiếu phải đã durable.

13. Sửa, xóa mềm và compact

13.1. Update một ván

Marker `recovery/update.properties` chứa Game ID, chiều dài payload trước update và toàn bộ old index record.

38. Đọc old record.
39. Ghi update marker.
40. Encode và append payload mới.
41. Force payload.
42. Ghi index record mới cùng Game ID.
43. Force index.
44. Xóa marker.

Nếu lỗi hoặc crash: restore old record, truncate payload về checkpoint và xóa marker.

13.2. Giới hạn update

Luồng replace hiện tại chủ yếu phục vụ movetext/annotation. Nó giữ nguyên IDs, packedDate, packedElo, result/ECO và deleted flag; chỉ đổi offset/length, các flags nội dung, ply count, semantic hash và CRC. Không dùng luồng này để thay toàn bộ header PGN.

13.3. Compact

45. Tạo bundle mới.
46. Quét record active.
47. Load PGN hoàn chỉnh.
48. Import qua writer chuẩn.
49. Copy sound file còn được tham chiếu.
50. Validate database mới.
51. Đóng cả hai bundle.
52. Atomic rename/swap ở mức thư mục nếu hỗ trợ.
53. Giữ backup database cũ.

Game ID sau compact: Import lại có thể tạo Game ID mới. Muốn giữ Game ID phải thiết kế tool chuyên biệt, giữ deleted slots hoặc cung cấp bảng ánh xạ.

14. Validation và an toàn dữ liệu

14.1. Checklist validator tối thiểu

Khu vực	Kiểm tra bắt buộc
Manifest	Tồn tại; format/version đúng; record size 56
Index header	Magic/version/header/record size; gameCount không âm; file

Khu vực	Kiểm tra bắt buộc
	không bị cắt
Index record	Offset/length hợp lệ; ID tồn tại; CRC khớp; plyCount không âm
Dictionary	Magic/version; count; varint; suffix; prefix UTF-16; ID không lặp
Payload	PCG1/version/codec; boundary string/tag; MOVE16 count; zlib length
Sound	Basename an toàn; extension; file tồn tại nếu cần phát

14.2. Giới hạn tài nguyên khuyến nghị

- Giới hạn payloadLength theo cấu hình.
- Giới hạn rawUtf8Length trước inflate.
- Giới hạn số custom tags và chiều dài mỗi string.
- Giới hạn moveCount theo payload boundary.
- Dùng phép cộng kiểm tra overflow cho offset + length.
- Decode UTF-8 nghiêm ngặt hoặc công bố rõ chính sách thay thế.

14.3. Sao lưu

Sao lưu đúng nghĩa là sao lưu toàn bộ thư mục `.pcod`, bao gồm manifest, index, dictionary, payload, sidecar, sound và recovery marker hiện có. Không chỉ copy ba file core nếu người dùng cần ghi chú hoặc âm thanh.

15. Chuyển đổi định dạng

15.1. PCOD sang định dạng khác

54. Đọc manifest.
55. Recovery hoặc từ chối nếu marker chưa xử lý.
56. Đọc dictionary.
57. Lặp Game ID từ 1 đến committed count.
58. Bỏ deleted nếu không yêu cầu.
59. Kiểm tra CRC.
60. Decode payload.
61. Dựng canonical PGN.
62. Chuyển domain object sang định dạng đích.

15.2. Định dạng khác sang PCOD

Phương án an toàn nhất cho phiên bản đầu tiên là:

```
Nguồn khác -> PGN chuẩn -> import bằng PCOD core
```

Nếu tự tạo binary trực tiếp, phải triển khai đầy đủ manifest, dictionary, codec, CRC, semantic hash, packing, atomic files, import recovery và crash simulation. Không copy index record giữa hai database vì name IDs sẽ trở sai dictionary.

16. Python skeleton đọc index

Đoạn mã sau minh họa đọc-only. Nó chưa phải writer hoàn chỉnh và chưa decode dictionary/payload.

```

from dataclasses import dataclass
from pathlib import Path
import struct
import zlib

PCI_MAGIC = 0x50434931
INDEX_HEADER_SIZE = 64
INDEX_RECORD_SIZE = 56

@dataclass(frozen=True)
class IndexRecord:
    payload_offset: int
    payload_length: int
    white_id: int
    black_id: int
    event_id: int
    site_id: int
    packed_date: int
    packed_elo: int
    packed_meta: int
    ply_count: int
    semantic_hash: int
    payload_crc: int

    @property
    def deleted(self) -> bool:
        return bool(self.packed_meta & (1 << 12))

RECORD_STRUCT = struct.Struct("<qiiiiiiiQI")

def read_index_header(path: Path) -> int:
    with path.open("rb") as f:
        raw = f.read(INDEX_HEADER_SIZE)
    if len(raw) != INDEX_HEADER_SIZE:
        raise ValueError("games.pci header bị cắt")
    magic, version, record_size, header_size, _ = struct.unpack_from(
        "<IHHII", raw, 0
    )
    game_count = struct.unpack_from("<Q", raw, 16)[0]
    if magic != PCI_MAGIC:
        raise ValueError("Sai PCI magic")
    if version != 1 or record_size != 56 or header_size != 64:
        raise ValueError("Index không tương thích")
    return game_count

def read_index_record(path: Path, game_id: int) -> IndexRecord:
    if game_id < 1:
        raise ValueError("Game ID phải bắt đầu từ 1")
    offset = INDEX_HEADER_SIZE + (game_id - 1) * INDEX_RECORD_SIZE
    with path.open("rb") as f:
        f.seek(offset)
        raw = f.read(INDEX_RECORD_SIZE)
    if len(raw) != INDEX_RECORD_SIZE:
        raise ValueError("Index record bị cắt")
    return IndexRecord(*RECORD_STRUCT.unpack(raw))

def read_payload(path: Path, record: IndexRecord) -> bytes:
    if record.payload_offset < 0 or record.payload_length < 0:
        raise ValueError("Offset/length không hợp lệ")
    with path.open("rb") as f:
        f.seek(record.payload_offset)
        data = f.read(record.payload_length)
    if len(data) != record.payload_length:
        raise ValueError("Payload bị cắt")
    if zlib.crc32(data) & 0xFFFFFFFF != record.payload_crc:
        raise ValueError("Payload CRC không khớp")
    return data

```

Unsigned semantic hash: Python `q` là signed int64. Khi hiển thị semantic hash theo dạng unsigned, dùng `value & 0xFFFFFFFFFFFFFFFF`.

17. Cheat sheet và xử lý lỗi

17.1. Mixed-endian cheat sheet

Thành phần	Endianness / encoding
games.pci header	Little-endian
games.pci record	Little-endian
names.pcn fixed fields	Big-endian
names.pcn varint	7-bit, low groups first
names.pcn suffix	UTF-8
PCG1 fixed header	Big-endian
PCG1 string lengths	Varint
MOVE16 code	Low byte rồi high byte
DEFLATE lengths	Varint
Compressed body	zlib stream

17.2. Lỗi thường gặp

Hiện tượng	Nguyên nhân thường gặp	Cách xử lý
ID/offset vô lý	Đọc sai endian	Đối chiếu cheat sheet; decode theo từng file
Tên lỗi hoặc prefix vượt chuỗi	Đếm prefix theo code point/byte	Tái hiện UTF-16 code units của Java
CRC sai	Đọc nhầm boundary hoặc file hỏng	Không decode; báo integrity error
Gzip không mở được	Payload là zlib, không phải gzip	Dùng zlib wrapper
Mất comment/variation	Ép dùng MOVE16	Fallback PGN_DEFLATE
Game ID lệch	Đếm record từ 0 hoặc tái sử dụng ID	Game ID = ordinal + 1; giữ deleted slot
Copy record sang DB khác sai tên	Name IDs phụ thuộc dictionary	Dùng PGN rồi import lại
File payload phình to	Orphan sau update	Compact qua bundle mới

17.3. Bất biến phải giữ

- Index header 64 byte, record 56 byte.
- Game ID ổn định và count là commit point.
- Delete chỉ bật flag.
- Payload append-only.
- Dictionary IDs explicit và ổn định.
- Section order PLAYER, EVENT, SITE.
- CRC trên toàn payload.
- MOVE16 chỉ khi round-trip an toàn.
- FEN lỗi phải được bảo toàn bằng PGN_DEFLATE.
- Sound reference trong PGN, nội dung trong `sound/`.
- H2 không phải nguồn core.
- Writer luôn có recovery marker.

18. Tương thích tương lai

- Từ chối formatVersion chưa hỗ trợ thay vì đoán.
- Có thể bỏ qua reserved fields/bits nếu không làm thay đổi boundary; nên cảnh báo khi lạ.
- Không yêu cầu reserved phải bằng 0 để đọc.
- Không xóa file chưa biết khi sao lưu hoặc di chuyển bundle.
- Writer 0.2 không thêm field vào record 56 byte.
- Metadata mới nên dùng payload custom tags, sidecar có version hoặc formatVersion mới.
- Khi format đổi, phát hành tài liệu mới như `PCOD\_DATABASE\_FORMAT\_0.3\_FOR\_CHATGPT.md` và giữ tài liệu 0.2 cho database cũ.

18.1. Checklist trước khi phát hành reader/writer

Nhóm	Checklist
Reader	Version; endian; CRC; limits; soft delete; UTF-16 prefix; sound sidecar
Writer	Recovery; force order; atomic dictionary; per-game rollback; round-trip tests
Kiểm thử	Unicode; FEN lỗi; variation; NAG; sound; database lớn; crash tại mọi commit stage
Phát hành	Không đổi record size; không tái sử dụng ID; giữ unknown files; có backup

19. Giấy phép MIT

Tác giả và chủ sở hữu bản quyền cho phép sử dụng tài liệu đặc tả này theo MIT License. Bản tiếng Anh dưới đây là nội dung giấy phép có hiệu lực; phần giải thích tiếng Việt chỉ nhằm hỗ trợ đọc hiểu.

19.1. Tóm tắt dễ hiểu

- Được sử dụng, sao chép, sửa đổi, hợp nhất, xuất bản, phân phối, cấp phép lại và bán bản sao tài liệu.
- Phải giữ thông báo bản quyền và thông báo giấy phép trong các bản sao hoặc phần quan trọng.
- Tài liệu được cung cấp “nguyên trạng”, không có bảo hành.
- Tác giả/chủ sở hữu không chịu trách nhiệm cho thiệt hại phát sinh từ việc sử dụng tài liệu.

19.2. Toàn văn MIT License

MIT License

Copyright (c) 2026 SpiderPawn Việt Nam

Permission is hereby granted, free of charge, to any person obtaining a copy of this documentation and associated specification files (the "Documentation"), to deal in the Documentation without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Documentation, and to permit persons to whom the Documentation is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Documentation.

THE DOCUMENTATION IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE DOCUMENTATION OR THE USE OR OTHER DEALINGS IN THE DOCUMENTATION.

Kết thúc tài liệu.